

Fundamentals of Object-Oriented Programming

Case Study & Technical Implementation: Bank Account Management System

1. Question & Problem Statement

Topic Context: Object-Oriented Paradigm, Objects & Classes, Data Abstraction & Encapsulation, Inheritance, Polymorphism, and Dynamic Binding.

Sample Case Study (CO1): Bank Account Management System

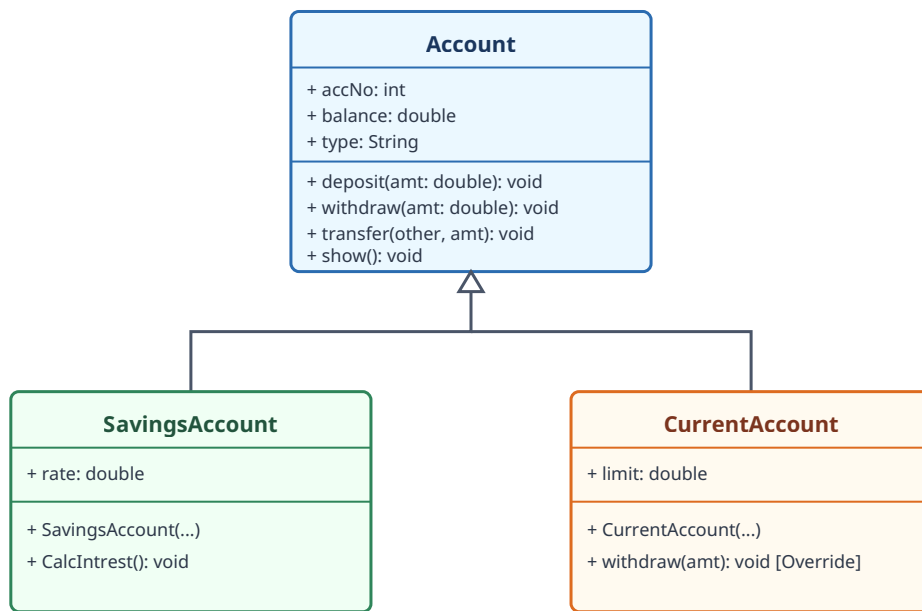
Problem Statement: Design and implement a system to manage customer bank accounts using Object-Oriented Programming principles.

- **Classes Required:** Account (Base Class), SavingsAccount (Derived Class), CurrentAccount (Derived Class), and Banksystem (Main Execution Class).
- **Attributes:** Account number (accNo), Balance (balance), Account type (type), Interest rate (rate for Savings), Overdraft limit (limit for Current).
- **Methods:** deposit(), withdraw(), transfer(), CalcIntrest(), show().
- **Object Usage:** Create objects for each customer's account type and execute operations such as deposits, withdrawals, interest calculations, and funds transfers via an interactive user menu.

2. Object-Oriented Concepts Map

OOP Paradigm	Implementation in Case Study
Classes & Objects	Classes like Account, SavingsAccount, and CurrentAccount act as blueprints. sAcc and cAcc are instantiated objects.
Inheritance	SavingsAccount and CurrentAccount extend the parent Account class, inheriting attributes (accNo, balance) and methods.
Polymorphism / Method Overriding	CurrentAccount overrides the withdraw() method of Account to implement overdraft capability tailored to current accounts.
Abstraction & Encapsulation	Account details and operational logic are bundled into cohesive class units, exposing clear operational methods while managing internal state changes.

3. UML Class Diagram



4. Step-by-Step Execution & Process Flow

Step 1: Base Class Definition (Account)

Defines generalized customer fields (`accNo`, `balance`, `type`). Includes fundamental banking operations like `deposit()`, standard `withdraw()`, `transfer()` to another account, and `show()` for displaying state.

Step 2: Specialization via Inheritance (Savings & Current Accounts)

- **SavingsAccount** inherits basic properties and adds interest calculation (`CalcIntrest()`) using interest rate (`rate`).
- **CurrentAccount** introduces an overdraft limit attribute (`limit`) allowing funds to be withdrawn up to `balance + limit`.

Step 3: Dynamic Method Overriding

CurrentAccount re-implements the `withdraw()` method to validate transactions against `(balance + limit)` rather than restricting to `balance` alone.

Step 4: Interactive Menu & Main Execution (Banksystem)

Instantiates a Savings Account (Acc 101, Bal 5000, Rate 5%) and a Current Account (Acc 201, Bal 3000, Limit 2000). A switch-case loop processes continuous transactions based on user choice until exit.

5. Complete Java Implementation

```
package record;
import java.util.Scanner;

// Base Class representing a generic Bank Account
class Account {
    int accNo;
    double balance;
    String type;

    Account(int no, double bal, String t) {
        accNo = no;
        balance = bal;
        type = t;
    }

    void deposit(double amt) {
        balance += amt;
        System.out.println("DEPOSITED:" + amt);
        System.out.println("BALANCE:" + balance);
    }

    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("WITHDRAW:" + amt);
            System.out.println("BALANCE:" + balance);
        } else {
            System.out.println("NOT ENOUGH BALANCE!");
        }
    }

    void transfer(Account other, double amt) {
        if (amt <= balance) {
            balance -= amt;
            other.balance += amt;
            System.out.println("TRANSFER DONE!");
        } else {
            System.out.println("TRANSFER FAILED!");
        }
    }

    void show() {
        System.out.println("Acc NO:" + accNo);
        System.out.println("TYPE:" + type);
        System.out.println("BALANCE:" + balance);
    }
}

// Derived Class for Savings Account
class SavingsAccount extends Account {
    double rate;

    SavingsAccount(int no, double bal, double r) {
        super(no, bal, "Savings");
        rate = r;
    }

    void CalcIntrest() {
```

```

        double intrest = balance * rate / 100;
        System.out.println("INTREST:" + intrest);
    }
}

// Derived Class for Current Account with Overdraft Support
class CurrentAccount extends Account {
    double limit;

    CurrentAccount(int no, double bal, double l) {
        super(no, bal, "CURRENT");
        limit = l;
    }

    @Override
    void withdraw(double amt) {
        if (amt <= balance + limit) {
            balance -= amt;
            System.out.println("WITHDRAW OK BALANCE IS:" + balance);
        } else {
            System.out.println("OVERDRAFT LIMIT CROSSED!");
        }
    }
}

// Main Execution Class
public class Banksystem {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        SavingsAccount sAcc = new SavingsAccount(101, 5000, 5);
        CurrentAccount cAcc = new CurrentAccount(201, 3000, 2000);
        int ch;

        do {
            System.out.println("
---BANK MENU---");
            System.out.println("1.DEPOSIT(SAVINGS)");
            System.out.println("2.WITHDRAW(SAVINGS)");
            System.out.println("3.INTREST(SAVINGS)");
            System.out.println("4.WITHDRAW(CURRENT)");
            System.out.println("5.TRANSFER SAVINGS->CURRENT");
            System.out.println("6.SHOW ACCOUNTS");
            System.out.println("7.EXIT");
            System.out.println("CHOICE");
            ch = s.nextInt();

            switch (ch) {
                case 1:
                    System.out.println("AMOUNT:");
                    sAcc.deposit(s.nextDouble());
                    break;
                case 2:
                    System.out.println("AMOUNT:");
                    sAcc.withdraw(s.nextDouble());
                    break;
                case 3:
                    sAcc.CalcIntrest();
                    break;
                case 4:
                    System.out.println("AMOUNT:");
                    cAcc.withdraw(s.nextDouble());

```

```

        break;
    case 5:
        System.out.println("AMOUNT");
        sAcc.transfer(cAcc, s.nextDouble());
        break;
    case 6:
        System.out.println("SAVINGS:");
        sAcc.show();
        System.out.println("CURRENT:");
        cAcc.show();
        break;
    case 7:
        System.out.println("BYE!");
        break;
    default:
        System.out.println("WRONG CHOICE!");
    }
} while (ch != 7);

s.close();
}
}

```